

# OSINT Blueprint

## OSINT PRODUCT BLUEPRINT

=====

PIPELINE: identifier -> normalize -> parallel source queries -> correlation graph -> confidence -> report  
 PIVOTING: every finding seeds new queries (email -> username -> name -> phone), max\_depth=2.

### TECHNIQUE MATRIX

- Email: registration enumeration (user-scanner / socialscan), breaches (HIBP, DeHashed), Gravatar JSON, Google-account surface (Epieos/GHunt)
- Phone: E.164 + carrier/line-type (phonenumbers), enrichment (Twilio Lookup, Numverify), WhatsApp/Telegram presence, breach records
- Name: search-engine x-ray (Serper, site: dorks), people-data APIs (PDL/Apollo)
- Username: Mairret (3000+ sites), Sherlock, Blackbird

BREACHES: HIBP v3 via hibp-api-key (free at haveibeenpwned.com). 200 = breaches found, 404 = clean. Add more engines (DeHashed/Snubase/IntelX) behind the same Source API.

### WEB APP (FastAPI + Jinja2)

```
python run_web.py      -> http://127.0.0.1:8000 (opens browser automatically)
POST /search           -> runs the pipeline, saves report, redirects to result page
                        (optional proxy field, or OSINT_PROXY env var)
GET /history           -> every saved search (open / pdf / gml links)
GET /report/{id}       -> results page (identifiers, findings, confidence, pivots)
GET /report/{id}/pdf   -> server-generated PDF download
GET /report/{id}/gml   -> correlation graph as GML (opens in Gephi / yEd)
Browser Print / Save-as-PDF -> printable layout via print CSS
Reports persist as JSON+GML under reports/ (recent on home page, full history page).
```

PROXY: per-request field on the search form, or OSINT\_PROXY=http://user:pass@host:port environment variable. Applies to every httpx-backed live source via the shared client. Subprocess-only sources (maigret, socialscan) bypass it; run those over a system/WSL proxy.

### CONFIDENCE MODEL

VERIFIED - target server explicitly confirmed (HTTP 200 profile, reset-flow response)  
 LIKELY - corroborated search hit / single source  
 UNSURE - collision-prone (esp. names)

ENGINEERING: async httpx + per-source rate limits, proxy rotation, module health-checks (module rot is the #1 failure mode), graph persistence (JSON + GML), Redis caching.

COMPLIANCE: GDPR/CCPA controller duties, FCRA ToS prohibition on employment/credit/tenant screening, platform-ToS review of enumeration modules. For authorized use only.

### USAGE

```
pip install -r requirements.txt
pipx install maigret          # optional username sweep (3000+ sites)
set HIBP_API_KEY=...          # optional breach module
set SERPER_API_KEY=...        # optional search x-ray
set OSINT_PROXY=http://user:pass@host:port # optional proxy for live lookups
python run_web.py             # start the web app
python -m osint "john.doe@gmail.com" # or CLI mode
python build_pdf.py           # regenerate this document
```

**osint/models.py**

```

from __future__ import annotations
from dataclasses import dataclass, field
from enum import Enum
import time

class IdentifierType(str, Enum):
    EMAIL = "email"
    PHONE = "phone"
    USERNAME = "username"
    NAME = "name"
    DOMAIN = "domain"

class Confidence(str, Enum):
    VERIFIED = "verified" # target server explicitly confirmed existence
    LIKELY = "likely" # >=2 independent sources corroborate
    UNSURE = "unsure" # single weak source / collision risk

@dataclass(frozen=True)
class Identifier:
    type: IdentifierType
    value: str

    def key(self) -> str:
        return f"{self.type.value}:{self.value.strip().lower()}"

    def __str__(self) -> str:
        return f"{self.type.value}:{self.value}"

    def to_dict(self) -> dict:
        return {"type": self.type.value, "value": self.value}

    @classmethod
    def from_dict(cls, data: dict) -> "Identifier":
        return cls(IdentifierType(data["type"]), data["value"])

@dataclass
class Finding:
    source: str
    identifier: Identifier
    confidence: Confidence
    url: str | None = None
    details: dict = field(default_factory=dict)
    pivots: list = field(default_factory=list)
    discovered: float = field(default_factory=time.time)

    def to_dict(self) -> dict:
        """Plain-dict form suitable for JSON caching / serialization."""
        return {
            "source": self.source,
            "identifier": self.identifier.to_dict(),
            "confidence": self.confidence.value,
            "url": self.url,
            "details": self.details,
            "pivots": [p.to_dict() for p in self.pivots],
            "discovered": self.discovered,
        }

    @classmethod
    def from_dict(cls, data: dict) -> "Finding":
        return cls(
            source=data["source"],
            identifier=Identifier.from_dict(data["identifier"]),
            confidence=Confidence(data["confidence"]),
            url=data.get("url"),
            details=data.get("details", {}),
            pivots=[Identifier.from_dict(p) for p in data.get("pivots", [])],
            discovered=data.get("discovered", time.time()),
        )

```

**osint/normalizers.py**

```

from __future__ import annotations
import hashlib
import re

import phonenumbers
from phonenumbers import NumberParseException, PhoneNumberFormat

from .models import Identifier, IdentifierType

def normalize_email(raw: str) -> Identifier:
    value = raw.strip().lower()
    if not re.match(r"^[^@\s]+@[^@\s]+\.[^@\s]+$", value):
        raise ValueError(f"invalid email: {raw}")
    return Identifier(IdentifierType.EMAIL, value)

def normalize_phone(raw: str, default_region: str = "US") -> Identifier:
    try:
        num = phonenumbers.parse(raw, default_region)
    except NumberParseException as exc:
        raise ValueError(f"invalid phone: {raw}") from exc
    if not phonenumbers.is_valid_number(num):
        raise ValueError(f"phone not valid: {raw}")
    return Identifier(IdentifierType.PHONE, phonenumbers.format_number(num, PhoneNumberFormat.E164))

def _name_parts(name: str) -> list:
    parts = [re.sub(r"^[a-z0-9]", "", p.lower()) for p in name.split()]
    return [p for p in parts if p]

def username_candidates(name: str) -> list:
    """jdoe, johndoe, john.doe, john-doe, doe.john, j_doe ..."""
    parts = _name_parts(name)
    if not parts:
        return []
    first, last = parts[0], parts[-1]
    cands = {
        first + last, first + "." + last, first + "-" + last,
        first[0] + last, first[0] + "." + last, first + "_" + last,
        last + first, last + "." + last, first, last,
    }
    return sorted(c for c in cands if len(c) >= 3)

def email_permutations(name: str, domains=("gmail.com", "outlook.com", "yahoo.com")) -> list:
    parts = _name_parts(name)
    if len(parts) < 2:
        return []
    f, l = parts[0], parts[-1]
    patterns = {"{f}.{l}", "{f}{l}", "{f[0]}{l}", "{f[0]}.{l}"
    return [
        Identifier(IdentifierType.EMAIL, f"{p}@{d}")
        for p in patterns if len(p) >= 4
        for d in domains
    ]

def gravatar_hash(email: str) -> str:
    return hashlib.md5(email.strip().lower().encode()).hexdigest()

_DOMAIN_RE = re.compile(r"^(?!-)[a-z0-9-]{1,63}(?!-)(\.(?!-)[a-z0-9-]{1,63}(?!-))*\.[a-z]{2,24}$")

def looks_like_domain(raw: str) -> bool:
    """Heuristic only ? a two-label lowercase token with a plausible TLD.

    Deliberately conservative: usernames like "john.doe" also match this
    shape, so callers should check this *before* falling back to USERNAME,
    not instead of letting the user force a type explicitly.
    """

```

```
value = raw.strip().lower()
return "." in value and " " not in value and bool(_DOMAIN_RE.match(value))
```

```
def normalize_domain(raw: str) -> Identifier:
    value = raw.strip().lower()
    if not looks_like_domain(value):
        raise ValueError(f"invalid domain: {raw}")
    return Identifier(IdentifierType.DOMAIN, value)
```

**osint/graph.py**

```

from __future__ import annotations
import networkx as nx

from .models import Finding

class CorrelationGraph:
    def __init__(self):
        self.g = nx.Graph()
        self.findings: list = []

    def add_finding(self, f: Finding) -> None:
        self.findings.append(f)
        src = f"src:{f.source}"
        ident_key = f.identifier.key()
        if not self.g.has_node(ident_key):
            self.g.add_node(ident_key, **{"type": f.identifier.type.value,
                                           "value": f.identifier.value,
                                           "label": "identifier"})

        if not self.g.has_node(src):
            self.g.add_node(src, **{"label": "source"})
        self.g.add_edge(ident_key, src, kind="observed_in", confidence=f.confidence.value)

        for pivot in f.pivots:
            pkey = pivot.key()
            if not self.g.has_node(pkey):
                self.g.add_node(pkey, **{"type": pivot.type.value, "value": pivot.value,
                                           "label": "identifier"})

            # edge weight accumulates corroboration across sources
            if self.g.has_edge(ident_key, pkey):
                self.g[ident_key][pkey]["weight"] += 1
            else:
                self.g.add_edge(ident_key, pkey, kind="co-occurrence", weight=1)

    def score(self, key: str) -> int:
        """Corroboration = sum of edge weights (multiple independent hits = strong)."""
        if key not in self.g:
            return 0
        return sum(d.get("weight", 1) for _, _, d in self.g.edges(key, data=True))

    def save(self, path: str) -> None:
        nx.write_gml(self.g, path)

```

**osint/confidence.py**

```
from .models import Confidence, IdentifierType

def classify(finding_count: int, independent_sources: int, ident_type: str) -> Confidence:
    if ident_type == IdentifierType.NAME.value:
        # names are inherently collision-prone: require strong corroboration
        return Confidence.VERIFIED if independent_sources >= 3 else Confidence.LIKELY
    if finding_count >= 1 and independent_sources >= 2:
        return Confidence.VERIFIED
    return Confidence.LIKELY if finding_count else Confidence.UNSURE
```

## osint/reporting.py

```

from __future__ import annotations
import io
from collections import defaultdict

import networkx as nx

def render_markdown(graph) -> str:
    """Render a human-readable markdown report from the correlation graph."""
    lines = ["# OSINT Report", ""]
    by_type = defaultdict(list)
    for n, d in graph.g.nodes(data=True):
        if d.get("label") == "identifier":
            by_type[d["type"]].append((n, d))
    for t, nodes in sorted(by_type.items()):
        if not nodes:
            continue
        lines += [f"## {t.upper()} ({len(nodes)})", ""]
        lines += ["| Identifier | Corroboration |", "|---|---|"]
        for key, d in sorted(nodes, key=lambda x: -graph.score(x[0])):
            lines.append(f"| {d['value']} | {graph.score(key)} |")
        lines.append("")
    lines += ["## Findings", ""]
    for u, v, d in graph.g.edges(data=True):
        if d.get("kind") == "observed_in":
            lines.append(f"- {u} observed via {v} [{d['confidence']}]")
    return "\n".join(lines)

def graph_to_dict(graph) -> dict:
    """Serialize the correlation graph + findings into a JSON-safe dict."""
    identifiers = []
    for n, d in graph.g.nodes(data=True):
        if d.get("label") == "identifier":
            identifiers.append({"key": n, "type": d["type"], "value": d["value"],
                               "score": graph.score(n)})
    findings = []
    for f in graph.findings:
        findings.append({
            "source": f.source,
            "identifier": {"key": f.identifier.key(), "type": f.identifier.type.value,
                          "value": f.identifier.value},
            "confidence": f.confidence.value,
            "url": f.url,
            "details": f.details,
        })
    pivots = []
    for u, v, d in graph.g.edges(data=True):
        if d.get("kind") == "co-occurrence":
            pivots.append({"from": u, "to": v, "weight": d.get("weight", 1)})
    return {"identifiers": identifiers, "findings": findings, "pivots": pivots}

def _write_gml(g) -> str:
    """networkx GML writer needs a binary stream; return the decoded text."""
    buf = io.BytesIO()
    nx.write_gml(g, buf)
    return buf.getvalue().decode("ascii")

def graph_to_gml(graph) -> str:
    """Serialize the live correlation graph as GML text (Gephi / yEd compatible)."""
    return _write_gml(graph.g)

def graph_dict_to_gml(data: dict) -> str:
    """Fallback: rebuild a networkx graph from a stored graph dict, emit GML."""
    g = nx.Graph()
    for i in data.get("identifiers", []):
        g.add_node(i["key"], label="identifier", type=i.get("type", "identifier"),
                  value=i.get("value", ""), score=i.get("score", 0))
    for f in data.get("findings", []):
        ident = f.get("identifier") or {}

```

```
src = f"src:{f.get('source', '?')}"
ident_key = ident.get("key")
if ident_key is None:
    continue
if not g.has_node(ident_key):
    g.add_node(ident_key, label="identifier", type=ident.get("type", "identifier"),
               value=ident.get("value", ident_key), score=0)
if not g.has_node(src):
    g.add_node(src, label="source")
g.add_edge(ident_key, src, kind="observed_in",
           confidence=f.get("confidence", "unsure"))
for p in data.get("pivots", []):
    g.add_edge(p["from"], p["to"], kind="co-occurrence",
              weight=p.get("weight", 1))
try:
    return _write_gml(g)
except Exception:
    return ""
```

**osint/orchestrator.py**

```

from __future__ import annotations
import asyncio
import logging
import os

import httpx

from .graph import CorrelationGraph
from .models import Identifier, IdentifierType
from .normalizers import username_candidates
from .sources import REGISTRY
from .sources.wrappers import MaignetSource, SocialscanSource
from .sources.dorks import SerperDorkSource
from .sources.telegram_phone import TelegramPhoneSource
from .sources.breaches import HibpSource

log = logging.getLogger("osint")

ACTIVE_SOURCES = REGISTRY + [MaignetSource(), SocialscanSource(),
                              SerperDorkSource(), TelegramPhoneSource(),
                              HibpSource()]

UA = {"User-Agent": "Mozilla/5.0 (X11; Linux x86_64) osint/0.1"}

def expand(identifier: Identifier) -> list:
    """Pivot expansion: identifier -> candidate new identifiers."""
    if identifier.type == IdentifierType.NAME:
        return [Identifier(IdentifierType.USERNAME, u) for u in username_candidates(identifier.value)]
    return []

class Orchestrator:
    def __init__(self, max_depth: int = 2, proxy: str | None = None,
                 platform: str | None = None, include_sources: set[str] | None = None,
                 exclude_sources: set[str] | None = None):
        self.graph = CorrelationGraph()
        self.seen: set = set()
        self.max_depth = max_depth
        # A platform page uses a dedicated site-restricted Serper source while
        # preserving the other public and optional sources.
        sources = (REGISTRY + [MaignetSource(), SocialscanSource(),
                              SerperDorkSource(platform), TelegramPhoneSource(), HibpSource()])
        if platform else ACTIVE_SOURCES
        if include_sources:
            sources = [s for s in sources if s.name in include_sources]
        if exclude_sources:
            sources = [s for s in sources if s.name not in exclude_sources]
        self.sources = sources
        # Explicit proxy wins; otherwise fall back to OSINT_PROXY env var.
        self.proxy = proxy or os.environ.get("OSINT_PROXY") or None

    @staticmethod
    def _http2_available() -> bool:
        """httpx needs the optional `h2` package for http2=True; degrade to
        HTTP/1.1 instead of crashing when it's not installed."""
        try:
            import h2 # noqa: F401
            return True
        except ImportError:
            log.debug("h2 not installed ? falling back to HTTP/1.1 (pip install httpx[http2] to enable)")
            return False

    async def _client(self) -> httpx.AsyncClient:
        kwargs = dict(http2=self._http2_available(), follow_redirects=True, timeout=30, headers=UA)
        if self.proxy:
            kwargs["proxy"] = self.proxy
        try:
            return httpx.AsyncClient(**kwargs)
        except TypeError: # httpx < 0.26: use the legacy proxies mapping
            kwargs.pop("proxy")
            kwargs["proxies"] = {"all://": self.proxy}
        return httpx.AsyncClient(**kwargs)

```

```

async def run(self, seed: Identifier) -> CorrelationGraph:
    client = await self._client()
    try:
        await self._wave([seed], 0, client)
    finally:
        await client.aclose()
    return self.graph

async def _wave(self, batch: list, depth: int, client: httpx.AsyncClient):
    fresh = [i for i in batch if i.key() not in self.seen]
    self.seen.update(i.key() for i in fresh)
    if not fresh or depth > self.max_depth:
        return

    tasks = []
    for i in fresh:
        for s in self.sources:
            if s.enabled and i.type in s.handles:
                tasks.append(asyncio.create_task(s.safe_query(i, client)))

    next_wave: list = []
    if tasks:
        for task in asyncio.as_completed(tasks):
            for f in await task:
                log.info("hit: %s via %s [%s]", f.identifier, f.source, f.confidence.value)
                self.graph.add_finding(f)
                next_wave.extend(f.pivots)
    next_wave += [p for i in fresh for p in expand(i)]
    await self._wave(next_wave, depth + 1, client)

```

**osint/ \_\_main\_\_.py**

```

import argparse
import asyncio
import logging
import os
import sys
from pathlib import Path

from .models import Identifier, IdentifierType
from .normalizers import looks_like_domain, normalize_domain, normalize_email, normalize_phone
from .orchestrator import Orchestrator
from .reporting import render_markdown

try:
    from dotenv import load_dotenv
    load_dotenv() # picks up .env in the current directory, if present
except ImportError:
    pass # python-dotenv not installed ? env vars still work if set some other way

_PHONE_PUNCTUATION = str.maketrans("", "", "+-.( ) \t")

def _looks_like_phone(raw: str) -> bool:
    """Digits plus common phone punctuation (spaces, dashes, parens, dots),
    with a plausible digit count ? not just a bare numeric string."""
    stripped = raw.translate(_PHONE_PUNCTUATION)
    return stripped.isdigit() and 7 <= len(stripped) <= 15

def coerce(raw: str, forced_type: str | None = None) -> Identifier:
    raw = raw.strip()
    if forced_type:
        builders = {
            "email": normalize_email, "phone": normalize_phone,
            "domain": normalize_domain,
            "name": lambda v: Identifier(IdentifierType.NAME, v.title()),
            "username": lambda v: Identifier(IdentifierType.USERNAME, v.lower()),
        }
        return builders[forced_type](raw)
    if "@" in raw:
        return normalize_email(raw)
    if _looks_like_phone(raw):
        return normalize_phone(raw)
    if any(c in raw for c in " \t"):
        return Identifier(IdentifierType.NAME, raw.title())
    if looks_like_domain(raw):
        return normalize_domain(raw)
    return Identifier(IdentifierType.USERNAME, raw.lower())

def parse_args(argv: list) -> argparse.Namespace:
    p = argparse.ArgumentParser(
        prog="python -m osint",
        description="Pivot an identifier (email, phone, name, username, or domain) "
        "across OSINT sources and build a correlation report.",
    )
    p.add_argument("identifier", nargs="+",
        help="the seed identifier, e.g. an email, phone number, name, username, or domain")
    p.add_argument("--type", choices=["email", "phone", "name", "username", "domain"],
        default=None,
        help="force how the identifier is interpreted instead of auto-detecting "
        "(useful for ambiguous values like 'jane.doe')")
    p.add_argument("--max-depth", type=int, default=2,
        help="how many pivot hops to follow from the seed (default: 2)")
    p.add_argument("--sources", default=None,
        help="comma-separated list of source names to use, e.g. 'github,hibp' "
        "(default: all enabled sources)")
    p.add_argument("--exclude-sources", default=None,
        help="comma-separated list of source names to skip")
    p.add_argument("--platform", default=None,
        help="restrict the search-engine dork source to a specific site, e.g. 'linkedin.com'")
    p.add_argument("--proxy", default=None,
        help="proxy URL for outbound requests (overrides OSINT_PROXY env var)")

```

```

p.add_argument("--output", default=None,
               help="output file path prefix (default: 'report_<identifier-type>')")
p.add_argument("--no-cache", action="store_true",
               help="bypass the result cache for this run (same as OSINT_NO_CACHE=1)")
p.add_argument("-v", "--verbose", action="store_true", help="enable debug logging")
return p.parse_args(argv)

def main(argv: list | None = None) -> int:
    args = parse_args(sys.argv[1:] if argv is None else argv)
    logging.basicConfig(level=logging.DEBUG if args.verbose else logging.INFO,
                        format="%(asctime)s %(levelname)s %(message)s")

    if args.no_cache:
        os.environ["OSINT_NO_CACHE"] = "1"

    try:
        ident = coerce(" ".join(args.identifier), args.type)
    except ValueError as exc:
        print(f"error: {exc}", file=sys.stderr)
        return 1

    include = set(args.sources.split(",")) if args.sources else None
    exclude = set(args.exclude_sources.split(",")) if args.exclude_sources else None

    orch = Orchestrator(max_depth=args.max_depth, proxy=args.proxy,
                        platform=args.platform, include_sources=include,
                        exclude_sources=exclude)
    graph = asyncio.run(orch.run(ident))

    prefix = args.output or f"report_{ident.type.value}"
    out = Path(f"{prefix}.md")
    out.write_text(render_markdown(graph))
    graph.save(f"{prefix}.gml")
    print(f"[+] report: {out}")
    print(f"[+] graph: {prefix}.gml")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

**osint/sources/base.py**

```

from __future__ import annotations
import abc
import asyncio
import logging
import time

import httpx

from ..cache import get_cache
from ..models import Finding, Identifier

log = logging.getLogger("osint")

# Exceptions worth retrying: transient network hiccups, not logic bugs.
RETRYABLE = (httpx.TimeoutException, httpx.ConnectError, httpx.ReadError, asyncio.TimeoutError)

class Source(abc.ABC):
    name: str = "base"
    handles: set = set()
    enabled: bool = True
    min_interval: float = 0.0          # per-source rate limit (seconds)
    cache_ttl: float = 86400.0         # seconds; 0 disables caching for this source
    max_retries: int = 2               # additional attempts after the first, on RETRYABLE errors
    retry_backoff: float = 0.5         # seconds, doubled each retry
    _last_call: float = 0.0

    async def rate_limit(self) -> None:
        wait = self.min_interval - (time.monotonic() - self._last_call)
        if wait > 0:
            await asyncio.sleep(wait)
        self._last_call = time.monotonic()

    @abc.abstractmethod
    async def query(self, identifier: Identifier, client: httpx.AsyncClient) -> list:
        ...

    def _cache_key(self, identifier: Identifier) -> str:
        return f"{self.name}:{identifier.key()}"

    async def safe_query(self, identifier: Identifier, client: httpx.AsyncClient) -> list:
        """Never let one dead source kill the pipeline (module-rot defense).

        Also handles result caching and retries on transient (network/timeout)
        errors so a single flaky request doesn't drop a source's findings for
        the whole run.
        """
        key = self._cache_key(identifier)
        cached = get_cache().get(key, self.cache_ttl)
        if cached is not None:
            log.debug("[%s] cache hit for %s", self.name, identifier)
            return [Finding.from_dict(d) for d in cached]

        attempt = 0
        while True:
            try:
                await self.rate_limit()
                findings = await asyncio.wait_for(self.query(identifier, client), timeout=30)
                get_cache().set(key, [f.to_dict() for f in findings])
                return findings
            except RETRYABLE as exc:
                if attempt >= self.max_retries:
                    log.warning("[%s] giving up on %s after %d attempts: %s",
                                self.name, identifier, attempt + 1, exc)
                    return []
                delay = self.retry_backoff * (2 ** attempt)
                log.info("[%s] retryable error on %s (%s), retrying in %.1fs",
                        self.name, identifier, exc, delay)
                await asyncio.sleep(delay)
                attempt += 1
            except Exception as exc:
                # Non-transient (bad JSON, bug, unexpected shape, etc.) ? don't
                # retry, just log and move on so one source can't stall a run.

```

```
log.warning("[%s] failed on %s: %s", self.name, identifier, exc)
return []
```

**osint/sources/native.py**

```

from __future__ import annotations

import logging

import httpx
import phonenumbers
from phonenumbers import carrier, geocoder, timezone as tz

from ..models import Confidence, Finding, Identifier, IdentifierType
from ..normalizers import gravatar_hash
from .base import Source

log = logging.getLogger("osint")

class NativeSource(Source):
    """Shared plumbing: verified HTTP checks with confidence=VERIFIED."""

    def finding(self, identifier, url=None, **details):
        return Finding(source=self.name, identifier=identifier,
                       confidence=Confidence.VERIFIED, url=url, details=details)

class GravatarSource(NativeSource):
    name = "gravatar"
    handles = {IdentifierType.EMAIL}

    async def query(self, identifier, client):
        r = await client.get(f"https://en.gravatar.com/{gravatar_hash(identifier.value)}.json")
        if r.status_code != 200:
            return []
        entry = (r.json().get("entry") or [{}])[0]
        if not entry:
            return []
        f = self.finding(identifier, url=f"https://gravatar.com/{entry.get('preferredUsername', '')}",
                        display=entry.get("displayName"))
        if entry.get("preferredUsername"):
            f.pivots.append(Identifier(IdentifierType.USERNAME, entry["preferredUsername"]))
        if entry.get("name", {}).get("formatted"):
            f.pivots.append(Identifier(IdentifierType.NAME, entry["name"]["formatted"]))
        for u in entry.get("urls", []):
            f.details.setdefault("profile_urls", []).append(u.get("value"))
        return [f]

class GithubUserSource(NativeSource):
    name = "github"
    handles = {IdentifierType.USERNAME}

    async def query(self, identifier, client):
        r = await client.get(f"https://api.github.com/users/{identifier.value}")
        if r.status_code == 200:
            data = r.json()
            f = self.finding(identifier, url=data.get("html_url"),
                            created=data.get("created_at"), bio=data.get("bio"))
            if data.get("name"):
                f.pivots.append(Identifier(IdentifierType.NAME, data["name"]))
            if data.get("email"):
                f.pivots.append(Identifier(IdentifierType.EMAIL, data["email"]))
            return [f]
        return []

class RedditUserSource(NativeSource):
    name = "reddit"
    handles = {IdentifierType.USERNAME}

    async def query(self, identifier, client):
        r = await client.get(f"https://www.reddit.com/user/{identifier.value}/about.json",
                            headers={"User-Agent": "osint/0.1 research"})
        if r.status_code == 200:
            data = r.json().get("data", {})
            return [self.finding(identifier, url=f"https://reddit.com/u/{identifier.value}",

```

```

        created=data.get("created_utc"), karma=data.get("total_karma"))]]

    return []

class TelegramUsernameSource(NativeSource):
    name = "telegram"
    handles = {IdentifierType.USERNAME}

    async def query(self, identifier, client):
        r = await client.get(f"https://t.me/{identifier.value}")
        if r.status_code == 200 and "tgme_page_title" in r.text:
            return [self.finding(identifier, url=f"https://t.me/{identifier.value}")]
        return []

class PhoneMetaSource(NativeSource):
    name = "phone-meta"
    handles = {IdentifierType.PHONE}
    cache_ttl = 0 # pure local computation ? no point caching, no API to save quota on

    async def query(self, identifier, client):
        try:
            num = phonenumbers.parse(identifier.value, None)
        except phonenumbers.NumberParseException as exc:
            log.warning("[phone-meta] could not parse %s: %s", identifier, exc)
            return []

        tzfn = getattr(tz, "time_zones_for_number", None) or getattr(tz, "timezones_for_number", None)
        try:
            tzs = str(tzfn(num))
        except Exception:
            tzs = "n/a"

        f = self.finding(identifier,
                        country=geocoder.description_for_number(num, "en"),
                        carrier=carrier.name_for_number(num, "en"),
                        timezones=tsz,
                        line_type=str(phonenumbers.number_type(num)))

        return [f]

class DomainRdapSource(NativeSource):
    """Domain registration data via RDAP (WHOIS's structured successor).

    Uses rdap.org as a bootstrap redirector so we don't need a per-registry
    RDAP server list or an extra dependency (whois libraries are frequently
    unmaintained / scrape-based). No API key required.
    """
    name = "domain-rdap"
    handles = {IdentifierType.DOMAIN}

    async def query(self, identifier, client):
        r = await client.get(f"https://rdap.org/domain/{identifier.value}",
                            follow_redirects=True)
        if r.status_code != 200:
            return []
        data = r.json()
        registrar = next(
            (e.get("vcardArray", [None, []])[1]
             for e in data.get("entities", []))
            if "registrar" in e.get("roles", [])),
            None,
        )
        events = {e.get("eventAction"): e.get("eventDate") for e in data.get("events", [])}
        nameservers = [ns.get("ldhName") for ns in data.get("nameservers", []) if ns.get("ldhName")]
        f = self.finding(
            identifier,
            url=f"https://rdap.org/domain/{identifier.value}",
            registered=events.get("registration"),
            expires=events.get("expiration"),
            last_changed=events.get("last changed"),
            nameservers=nameservers,
            status=data.get("status", []),
        )
        return [f]

```

```
REGISTRY = [GravatarSource(), GithubUserSource(), RedditUserSource(),  
            TelegramUsernameSource(), PhoneMetaSource(), DomainRdapSource()]
```

**osint/sources/wrappers.py**

```

from __future__ import annotations
import asyncio
import json
import shutil
import sys
from pathlib import Path

import httpx

from ..models import Confidence, Finding, Identifier, IdentifierType
from .base import Source


class MaigretSource(Source):
    """Username sweep across 3000+ sites via the `maigret` CLI (pipx install maigret)."""
    name = "maigret"
    handles = {IdentifierType.USERNAME}

    async def query(self, identifier: Identifier, client: httpx.AsyncClient):
        # Prefer a normal PATH installation, but also support the executable
        # installed into this project's virtual environment.
        executable = shutil.which("maigret")
        local_executable = Path(sys.executable).with_name("maigret.exe")
        if not executable and local_executable.is_file():
            executable = str(local_executable)
        if not executable:
            return []
        proc = await asyncio.create_subprocess_exec(
            executable, identifier.value, "--json", "-", "--timeout", "15",
            stdout=asyncio.subprocess.PIPE, stderr=asyncio.subprocess.DEVNULL)
        out, _ = await asyncio.wait_for(proc.communicate(), timeout=180)
        try:
            data = json.loads(out.decode() or "{}")
        except json.JSONDecodeError:
            return []
        findings = []
        for site, info in data.items():
            status = info.get("status", {})
            if status.get("status") == "Exist":
                f = Finding(source=f"maigret:{site}", identifier=identifier,
                           confidence=Confidence.VERIFIED,
                           url=status.get("url"), details=info)
                findings.append(f)
        return findings


class SocialscanSource(Source):
    """Email+username availability across ~15 major platforms via `socialscan`."""
    name = "socialscan"
    handles = {IdentifierType.EMAIL, IdentifierType.USERNAME}

    async def query(self, identifier: Identifier, client: httpx.AsyncClient):
        if not shutil.which("socialscan"):
            return []
        proc = await asyncio.create_subprocess_exec(
            "socialscan", identifier.value, "--json",
            stdout=asyncio.subprocess.PIPE, stderr=asyncio.subprocess.DEVNULL)
        out, _ = await asyncio.wait_for(proc.communicate(), timeout=120)
        try:
            data = json.loads(out.decode() or "[]")
        except json.JSONDecodeError:
            return []
        results = data if isinstance(data, list) else data.get("results", [])
        return [Finding(source=f"socialscan:{r.get('platform')}", identifier=identifier,
                       confidence=Confidence.VERIFIED if r.get("registered") else Confidence.UNSURE,
                       url=r.get("profile_url"), details=r)
                for r in results if r.get("registered")]

```

**osint/sources/dorks.py**

```

from __future__ import annotations
import os
from urllib.parse import urlparse

import httpx

from ..models import Confidence, Finding, Identifier, IdentifierType
from .base import Source

# Request a deeper result set to surface less-prominent, publicly indexed
# accounts. Serper searches its full public-web index without a site allowlist.
RESULTS_PER_QUERY = 100

DORK = {
    IdentifierType.NAME:      '{q}',
    IdentifierType.USERNAME:  '{q}',
    # Email addresses and phone numbers are often indexed outside the three
    # social networks used in the original query. Use Serper's full index so
    # a report does not hide otherwise relevant, publicly indexed results.
    IdentifierType.EMAIL:     '{q}',
    IdentifierType.PHONE:     '{q}',
}

class SerperDorkSource(Source):
    """Search-engine x-ray. Needs SERPER_API_KEY (serper.dev, generous free tier)."""
    name = "serper-dorks"
    handles = {IdentifierType.NAME, IdentifierType.USERNAME,
                IdentifierType.EMAIL, IdentifierType.PHONE}
    min_interval = 1.0

    def __init__(self, site: str | None = None):
        self.site = site

    async def query(self, identifier: Identifier, client: httpx.AsyncClient):
        key = os.getenv("SERPER_API_KEY")
        if not key:
            return []
        findings = []
        query = DORK[identifier.type].format(q=identifier.value)
        queries = [f"{query} site:{self.site}" if self.site else query]
        for q in queries:
            r = await client.post("https://google.serper.dev/search",
                                  headers={"X-API-KEY": key, "Content-Type": "application/json"},
                                  json={"q": q, "num": RESULTS_PER_QUERY})
            if r.status_code != 200:
                continue
            for hit in r.json().get("organic", []):
                url = hit.get("link")
                domain = urlparse(url).netloc.removeprefix("www.") if url else "web"
                findings.append(Finding(
                    # Serper supplies the search result; the domain is the
                    # actual public source the investigator should assess.
                    source=f"web:{domain}", identifier=identifier,
                    confidence=Confidence.LIKELY, # search-engine hit = corroborate, don't trust
                    url=url,
                    details={"title": hit.get("title"), "snippet": hit.get("snippet")}))
        return findings

```

**osint/sources/telegram\_phone.py**

```
from __future__ import annotations
import os

from ..models import Confidence, Finding, Identifier, IdentifierType
from .base import Source

class TelegramPhoneSource(Source):
    """Phone -> Telegram existence via contact import. Requires TELEGRAM_API_ID and
    TELEGRAM_API_HASH env vars (my.telegram.org) and a one-time interactive login.
    Telethon is imported lazily so the core pipeline works without it installed."""
    name = "telegram-phone"
    handles = {IdentifierType.PHONE}

    async def query(self, identifier: Identifier, client):
        api_id, api_hash = os.getenv("TELEGRAM_API_ID"), os.getenv("TELEGRAM_API_HASH")
        if not (api_id and api_hash):
            return []
        from telethon import TelegramClient, functions, types # optional dep
        async with TelegramClient("osint_session", int(api_id), api_hash) as tg:
            result = await tg(functions.contacts.ImportContactsRequest(
                contacts=[types.InputPhoneContact(client_id=0, phone=identifier.value,
                                                    first_name="", last_name=")])
            if result.imported_contacts:
                c = result.imported_contacts[0]
                return [Finding(source="telegram", identifier=identifier,
                                confidence=Confidence.VERIFIED,
                                url=f"https://t.me/{identifier.value.lstrip('+')}",
                                details={"user_id": str(c.user_id)})]
        return []
```

**osint/sources/breaches.py**

```

from __future__ import annotations
import os
from urllib.parse import quote

from ..models import Confidence, Finding, Identifier, IdentifierType
from .base import Source

class HibpSource(Source):
    """Have I Been Pwned breach exposure. Needs HIBP_API_KEY (free at haveibeenpwned.com).
    HTTP 200 = email appears in known breaches; 404 = no public breaches."""
    name = "hibp"
    handles = {IdentifierType.EMAIL}
    min_interval = 2.0

    async def query(self, identifier: Identifier, client):
        key = os.getenv("HIBP_API_KEY")
        if not key:
            return []
        url = f"https://haveibeenpwned.com/api/v3/breachedaccount/{quote(identifier.value)}"
        headers = {"hibp-api-key": key, "User-Agent": "osint/0.1"}
        r = await client.get(url, headers=headers,
                             params={"truncateResponse": "true",
                                     "includeUnverified": "false"})

        if r.status_code == 200:
            names = r.json()
            return [Finding(source="hibp", identifier=identifier,
                           confidence=Confidence.VERIFIED,
                           url="https://haveibeenpwned.com",
                           details={"breaches": names})]

        if r.status_code == 429:
            return [] # HIBP rate limit: skip quietly, next run picks it up
        return []

```

**run\_web.py**

```
import threading
import webbrowser

import uvicorn

if __name__ == "__main__":
    threading.Timer(1.2, lambda: webbrowser.open("http://127.0.0.1:8000")).start()
    print("[+] OSINT Web running at http://127.0.0.1:8000 (Ctrl+C to stop)")
    uvicorn.run("webapp.server:app", host="127.0.0.1", port=8000, log_level="info")
```

**webapp/server.py**

```

from __future__ import annotations
import logging
import time
from pathlib import Path

from fastapi import FastAPI, Form, Request
from fastapi.responses import HTMLResponse, RedirectResponse, Response
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from dotenv import load_dotenv

from osint.__main__ import coerce
from osint.models import Identifier, IdentifierType
from osint.normalizers import normalize_email, normalize_phone
from osint.orchestrator import Orchestrator
from osint.reporting import graph_dict_to_gml

from .pdf_export import render_report_pdf
from .store import ReportStore

load_dotenv()
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")

BASE = Path(__file__).resolve().parent
ROOT = BASE.parent

app = FastAPI(title="OSINT Web", docs_url=None, redoc_url=None)
templates = Jinja2Templates(directory=str(BASE / "templates"))
app.mount("/static", StaticFiles(directory=str(BASE / "static")), name="static")
store = ReportStore(ROOT / "reports")

ORDER = ["email", "phone", "username", "name", "domain"]
PLATFORMS = {
    "linkedin.com": "LinkedIn", "x.com": "X", "instagram.com": "Instagram",
    "facebook.com": "Facebook", "tiktok.com": "TikTok", "github.com": "GitHub",
    "medium.com": "Medium", "pinterest.com": "Pinterest",
    "youtube.com": "YouTube", "reddit.com": "Reddit", "threads.net": "Threads",
    "bsky.app": "Bluesky", "twitch.tv": "Twitch", "snapchat.com": "Snapchat",
    "t.me": "Telegram", "tumblr.com": "Tumblr", "soundcloud.com": "SoundCloud",
    "kick.com": "Kick", "substack.com": "Substack",
}

def prepare_report(data: dict) -> dict:
    """Turn a stored report dict into template-ready view data."""
    graph = data["graph"]
    lookup = {i["key"]: i for i in graph["identifiers"]}

    groups: dict = {}
    for i in graph["identifiers"]:
        groups.setdefault(i["type"], []).append(i)
    groups = {k: sorted(v, key=lambda x: -x["score"]) for k, v in groups.items()}
    groups = {k: groups[k] for k in sorted(groups, key=lambda t: ORDER.index(t) if t in ORDER else 99)}

    findings = []
    for f in graph["findings"]:
        details = f.get("details") or {}
        breaches = details.get("breaches") if isinstance(details, dict) else None
        if isinstance(breaches, list):
            summary = ", ".join(str(b) for b in breaches[:8])
        elif isinstance(details, dict):
            summary = " ? ".join(str(v) for v in (details.get("title"), details.get("snippet"))) if v
        else:
            summary = ""
        findings.append({
            "identifier": f["identifier"]["value"],
            "source": f["source"],
            "confidence": f["confidence"],
            "url": f.get("url"),
            "details_summary": summary,
            "details": details,
        })
    findings.sort(key=lambda x: ("verified", "likely", "unsure").index(x["confidence"]))

```

```

        if x["confidence"] in ("verified", "likely", "unsure") else 3)

confidence_counts = {level: sum(1 for f in findings if f["confidence"] == level)
                      for level in ("verified", "likely", "unsure")}
source_names = sorted({f["source"] for f in findings})

pivots = []
for p in graph["pivots"]:
    a = lookup.get(p["from"])
    b = lookup.get(p["to"])
    pivots.append({"from": a["value"] if a else p["from"],
                  "to": b["value"] if b else p["to"],
                  "weight": p["weight"]})

return {
    "id": data["id"],
    "seed": data["seed"],
    "when": time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(data["created"])),
    "groups": groups,
    "findings": findings,
    "total": len(findings),
    "confidence_counts": confidence_counts,
    "source_names": source_names,
    "source_count": len(source_names),
    "identifier_count": len(graph["identifiers"]),
    "shows_details": any(f.get("details_summary") for f in findings),
    "pivots": pivots,
}

@app.get("/", response_class=HTMLResponse)
async def index(request: Request):
    return templates.TemplateResponse(request=request, name="index.html",
                                     context={"request": request,
                                             "recent": store.list_recent()})

@app.get("/history", response_class=HTMLResponse)
async def history(request: Request):
    return templates.TemplateResponse(request=request, name="history.html",
                                     context={"request": request,
                                             "items": store.list_all()})

@app.get("/platforms", response_class=HTMLResponse)
async def platforms(request: Request):
    return templates.TemplateResponse(request=request, name="platforms.html",
                                     context={"request": request,
                                             "platforms": PLATFORMS})

def coerce_as(raw: str, identifier_type: str) -> Identifier:
    """Parse an input from a type-specific search form."""
    raw = raw.strip()
    try:
        kind = IdentifierType(identifier_type)
    except ValueError as exc:
        raise ValueError("unknown search type") from exc
    if kind == IdentifierType.EMAIL:
        return normalize_email(raw)
    if kind == IdentifierType.PHONE:
        return normalize_phone(raw)
    if kind == IdentifierType.NAME:
        if not raw or not any(char.isalpha() for char in raw):
            raise ValueError("enter a name")
        return Identifier(kind, raw.title())
    if kind == IdentifierType.USERNAME:
        if not raw or any(char.isspace() for char in raw):
            raise ValueError("enter a username without spaces")
        return Identifier(kind, raw.lower())
    return Identifier(kind, raw.lower())

@app.post("/search")
async def search(q: str = Form(...), search_type: str = Form("auto"), proxy: str = Form(""),

```

```

        platform: str = Form(""))):
    q = q.strip()
    try:
        ident = coerce(q) if search_type == "auto" else coerce_as(q, search_type)
    except ValueError as exc:
        return HTMLResponse(f"<p>Invalid input: {exc}</p><p><a href='/'>Back</a></p>",
                             status_code=400)
    if platform and platform not in PLATFORMS:
        return HTMLResponse("<p>Unknown platform.</p><p><a href='/platforms'>Back</a></p>",
                             status_code=400)
    graph = await Orchestrator(max_depth=2, proxy=proxy.strip() or None,
                               platform=platform or None).run(ident)
    rid = store.create(ident, graph)
    return RedirectResponse(f"/report/{rid}", status_code=303)

@app.get("/report/{rid}", response_class=HTMLResponse)
async def report(request: Request, rid: str):
    data = store.load(rid)
    if not data:
        return HTMLResponse("<p>Report not found.</p><p><a href='/'>Back</a></p>", status_code=404)
    return templates.TemplateResponse(request=request, name="report.html",
                                     context={"request": request, **prepare_report(data)})

@app.get("/report/{rid}/pdf")
async def report_pdf(rid: str):
    data = store.load(rid)
    if not data:
        return Response("Report not found", status_code=404)
    prepared = prepare_report(data)
    pdf_bytes = render_report_pdf(prepared)
    return Response(content=pdf_bytes, media_type="application/pdf",
                    headers={"Content-Disposition":
                             f'attachment; filename="osint_report_{rid}.pdf"'})

@app.get("/report/{rid}/gml")
async def report_gml(rid: str):
    data = store.load(rid)
    if not data:
        return Response("Report not found", status_code=404)
    gml = data.get("gml") or graph_dict_to_gml(data.get("graph") or {})
    if not gml.strip():
        return Response("Graph unavailable for this report", status_code=404)
    return Response(content=gml, media_type="application/octet-stream",
                    headers={"Content-Disposition":
                             f'attachment; filename="osint_graph_{rid}.gml"'})

```

**webapp/store.py**

```

from __future__ import annotations
import json
import time
import uuid
from pathlib import Path

from osint.reporting import graph_to_dict, graph_to_gml

class ReportStore:
    """Persists each search's correlation graph (JSON + GML) in <project>/reports/."""

    def __init__(self, directory: Path):
        self.directory = directory
        self.directory.mkdir(parents=True, exist_ok=True)

    def create(self, ident, graph) -> str:
        rid = uuid.uuid4().hex[:12]
        data = {"id": rid, "seed": ident.key(), "created": time.time(),
              "graph": graph_to_dict(graph), "gml": graph_to_gml(graph)}
        (self.directory / f"{rid}.json").write_text(
            json.dumps(data, indent=2, default=str, encoding="utf-8"))
        return rid

    def load(self, rid: str):
        p = self.directory / f"{rid}.json"
        if not p.exists():
            return None
        return json.loads(p.read_text(encoding="utf-8"))

    def _row(self, data: dict) -> dict:
        total = len((data.get("graph") or {}).get("findings", []))
        return {"id": data["id"], "seed": data["seed"], "total": total,
              "when": time.strftime("%Y-%m-%d %H:%M",
                                   time.localtime(data["created"]))}

    def _items(self, limit: int | None = None) -> list:
        files = sorted(self.directory.glob("*.json"),
                       key=lambda f: f.stat().st_mtime, reverse=True)
        if limit:
            files = files[:limit]
        items = []
        for p in files:
            try:
                data = json.loads(p.read_text(encoding="utf-8"))
            except Exception:
                continue
            items.append(self._row(data))
        return items

    def list_recent(self, limit: int = 10) -> list:
        return self._items(limit=limit)

    def list_all(self) -> list:
        return self._items()

```

**webapp/pdf\_export.py**

```

from __future__ import annotations

from fpdf import FPDF

ORDER = ["email", "phone", "username", "name", "domain"]
MINT = (78, 155, 124)
INK = (24, 31, 31)
MUTED = (91, 105, 103)
LINE = (215, 224, 220)
PALE = (241, 247, 244)

def _safe(value) -> str:
    """Keep the built-in PDF fonts safe for arbitrary source text."""
    return (str(value) if value is not None else "").encode("cp1252", "replace").decode("cp1252")

class ReportPDF(FPDF):
    def header(self):
        self.set_font("Helvetica", "B", 8)
        self.set_text_color(*MUTED)
        self.cell(0, 5, "OSINT / INVESTIGATION BRIEF", new_x="LMARGIN", new_y="NEXT")
        self.set_draw_color(*LINE)
        self.line(self.l_margin, 13, self.w - self.r_margin, 13)
        self.set_text_color(*INK)

    def footer(self):
        self.set_y(-14)
        self.set_draw_color(*LINE)
        self.line(self.l_margin, self.get_y(), self.w - self.r_margin, self.get_y())
        self.set_font("Helvetica", "", 8)
        self.set_text_color(*MUTED)
        self.cell(0, 8, "Authorized investigations only | Public-source observations", align="L")
        self.cell(0, 8, f"Page {self.page_no()}", align="R")

def _section(pdf: ReportPDF, number: str, title: str, caption: str = "") -> None:
    pdf.ln(4)
    pdf.set_font("Helvetica", "B", 8)
    pdf.set_text_color(*MINT)
    pdf.cell(12, 6, number)
    pdf.set_font("Helvetica", "B", 14)
    pdf.set_text_color(*INK)
    pdf.cell(0, 6, title, new_x="LMARGIN", new_y="NEXT")
    if caption:
        pdf.set_font("Helvetica", "", 8)
        pdf.set_text_color(*MUTED)
        pdf.cell(0, 5, caption, new_x="LMARGIN", new_y="NEXT")
    pdf.ln(2)

def _metric(pdf: ReportPDF, x: float, label: str, value: str, note: str) -> None:
    y = pdf.get_y()
    pdf.set_fill_color(*PALE)
    pdf.set_draw_color(*LINE)
    pdf.rect(x, y, 42.5, 23, style="DF")
    pdf.set_xy(x + 3, y + 3)
    pdf.set_font("Helvetica", "B", 7)
    pdf.set_text_color(*MUTED)
    pdf.cell(36, 4, _safe(label).upper())
    pdf.set_xy(x + 3, y + 8)
    pdf.set_font("Helvetica", "", 18)
    pdf.set_text_color(*INK)
    pdf.cell(36, 8, _safe(value))
    pdf.set_xy(x + 3, y + 17)
    pdf.set_font("Helvetica", "", 7)
    pdf.set_text_color(*MUTED)
    pdf.cell(36, 3, _safe(note))

def _finding(pdf: ReportPDF, finding: dict) -> None:
    if pdf.get_y() > 245:
        pdf.add_page()

```

```

y = pdf.get_y()
pdf.set_fill_color(248, 250, 249)
pdf.set_draw_color(*LINE)
pdf.rect(pdf.l_margin, y, 180, 11, style="DF")
pdf.set_xy(pdf.l_margin + 4, y + 3)
pdf.set_font("Helvetica", "B", 9)
pdf.set_text_color(*INK)
pdf.cell(57, 5, _safe(finding["identifier"]))
pdf.set_font("Helvetica", "", 8)
pdf.set_text_color(*MUTED)
pdf.cell(55, 5, _safe(finding["source"]))
pdf.set_font("Helvetica", "B", 8)
pdf.set_text_color(*MINT if finding["confidence"] == "verified" else (173, 128, 42))
pdf.cell(55, 5, _safe(finding["confidence"]).upper())
pdf.set_xy(pdf.l_margin + 4, y + 14)
pdf.set_font("Helvetica", "", 8)
pdf.set_text_color(*INK)
detail = finding.get("details_summary") or "No additional source metadata."
pdf.multi_cell(172, 4, _safe(detail), new_x="LMARGIN", new_y="NEXT")
if finding.get("url"):
    pdf.set_text_color(*MINT)
    pdf.cell(0, 4, _safe(finding["url"]), new_x="LMARGIN", new_y="NEXT")
pdf.ln(4)

```

```
def render_report_pdf(prepared: dict) -> bytes:
```

```

pdf = ReportPDF()
pdf.set_auto_page_break(True, margin=18)
pdf.set_margins(15, 18, 15)
pdf.add_page()

pdf.set_font("Helvetica", "B", 24)
pdf.set_text_color(*INK)
pdf.cell(0, 12, "Public-source findings", new_x="LMARGIN", new_y="NEXT")
pdf.set_font("Helvetica", "", 10)
pdf.set_text_color(*MUTED)
pdf.cell(0, 6, f"Seed identifier: {_safe(prepared['seed'])}", new_x="LMARGIN", new_y="NEXT")
pdf.cell(0, 6, f"Generated: {_safe(prepared['when'])} | Report: {_safe(prepared['id'])}",
        new_x="LMARGIN", new_y="NEXT")
pdf.ln(5)

```

```

counts = prepared.get("confidence_counts", {})
metrics = [("Findings", str(prepared["total"]), "observations"),
           ("Identifiers", str(prepared.get("identifier_count", 0)), "linked nodes"),
           ("Sources", str(prepared.get("source_count", 0)), "distinct providers"),
           ("Verified", str(counts.get("verified", 0)), "explicit confirmations")]
start_x = pdf.l_margin
for index, (label, value, note) in enumerate(metrics):
    _metric(pdf, start_x + index * 45, label, value, note)
pdf.set_y(pdf.get_y() + 29)

```

```

pdf.set_fill_color(*PALE)
pdf.set_draw_color(*MINT)
pdf.rect(pdf.l_margin, pdf.get_y(), 180, 17, style="DF")
pdf.set_xy(pdf.l_margin + 4, pdf.get_y() + 3)
pdf.set_font("Helvetica", "B", 8)
pdf.set_text_color(*INK)
pdf.cell(33, 5, "ASSESSMENT SCOPE")
pdf.set_font("Helvetica", "", 8)
pdf.set_text_color(*MUTED)
pdf.multi_cell(140, 4, "Public-source observations available at scan time. A finding is not proof of identity; review the linked s

```

```

_section(pdf, "01", "Evidence posture", f"{prepared.get('source_count', 0)} source providers queried")
posture = [("VERIFIED", counts.get("verified", 0), "explicit server confirmation"),
           ("LIKELY", counts.get("likely", 0), "credible source or search hit"),
           ("UNSURE", counts.get("unsure", 0), "collision or weak signal")]

```

```

for index, (label, value, note) in enumerate(posture):
    x = pdf.l_margin + index * 60
    pdf.set_font("Helvetica", "B", 10)
    pdf.set_text_color(*MINT if label == "VERIFIED" else (173, 128, 42) if label == "LIKELY" else MUTED)
    pdf.set_xy(x, pdf.get_y())
    pdf.cell(22, 6, label)
    pdf.set_font("Helvetica", "", 14)
    pdf.set_text_color(*INK)
    pdf.cell(12, 6, str(value))

```

```

pdf.set_font("Helvetica", "", 7)
pdf.set_text_color(*MUTED)
pdf.set_xy(x, pdf.get_y() + 7)
pdf.multi_cell(52, 3, note)
pdf.ln(7)

source_names = prepared.get("source_names", [])
if source_names:
    pdf.set_font("Helvetica", "B", 8)
    pdf.set_text_color(*MUTED)
    pdf.cell(30, 5, "SOURCE COVERAGE")
    pdf.set_font("Helvetica", "", 8)
    pdf.set_text_color(*INK)
    pdf.multi_cell(145, 5, _safe(" | ".join(source_names)), new_x="LMARGIN", new_y="NEXT")

_section(pdf, "02", "Identifier map", "Ranked by corroboration")
for identifier_type in ORDER:
    items = prepared["groups"].get(identifier_type)
    if not items:
        continue
    pdf.set_font("Helvetica", "B", 8)
    pdf.set_text_color(*MINT)
    pdf.cell(30, 6, identifier_type.upper())
    pdf.set_font("Helvetica", "", 9)
    pdf.set_text_color(*INK)
    for item in items:
        pdf.cell(112, 6, _safe(item["value"]))
        pdf.set_text_color(*MUTED)
        pdf.cell(0, 6, f"corroboration {item['score']}", new_x="LMARGIN", new_y="NEXT")
        pdf.set_text_color(*INK)

_section(pdf, "03", "Evidence ledger", f"{prepared['total']} observation(s), ordered by confidence")
if prepared["findings"]:
    for finding in prepared["findings"]:
        _finding(pdf, finding)
else:
    pdf.set_font("Helvetica", "", 9)
    pdf.set_text_color(*MUTED)
    pdf.multi_cell(0, 5, "No findings were returned for this identifier.")

if prepared.get("pivots"):
    _section(pdf, "04", "Pivot trail", "Linked identifiers discovered during collection")
    for pivot in prepared["pivots"]:
        pdf.set_font("Helvetica", "", 9)
        pdf.set_text_color(*INK)
        pdf.cell(0, 6, f"{_safe(pivot['from'])} -> {_safe(pivot['to'])}")
        pdf.set_text_color(*MUTED)
        pdf.cell(0, 6, f" weight {pivot['weight']}", new_x="LMARGIN", new_y="NEXT")

return bytes(pdf.output())

```

## webapp/templates/index.html

```

<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>OSINT People Search</title>
<link rel="stylesheet" href="/static/style.css">
</head>
<body>
<div class="topbar">
  <a class="brand" href="/">OSINT</a>
  <nav class="actions">
    <a class="btn" href="platforms.html">Platforms</a>
    <a class="btn" href="history.html">History</a>
  </nav>
</div>
<main class="container">
  <header class="hero">
    <h1>OSINT</h1>
    <p class="tagline">Find every public trace of a name, email, phone number or username.</p>
  </header>
  <form class="search-box universal-search" method="post" action="/search">
    <input type="text" name="q" placeholder='e.g. "jane.doe@gmail.com", "+1 415 555 0123", "Jane Doe", "jdoe"' autofocus required>
    <input type="hidden" name="search_type" value="auto">
    <button type="submit">Search</button>
  </form>
  <p class="search-help muted">Or choose exactly what you want to search.</p>

  <section class="search-grid" aria-label="Type-specific searches">
    <form class="search-card" method="post" action="/search">
      <input type="hidden" name="search_type" value="email">
      <label for="email">Email address</label>
      <input id="email" type="email" name="q" placeholder="name@example.com" required>
      <button type="submit">Search email</button>
    </form>
    <form class="search-card" method="post" action="/search">
      <input type="hidden" name="search_type" value="phone">
      <label for="phone">Phone number</label>
      <input id="phone" type="tel" name="q" placeholder="+1 415 555 0123" required>
      <button type="submit">Search phone</button>
    </form>
    <form class="search-card" method="post" action="/search">
      <input type="hidden" name="search_type" value="username">
      <label for="username">Username</label>
      <input id="username" type="text" name="q" placeholder="jane_doe" required>
      <button type="submit">Search username</button>
    </form>
    <form class="search-card" method="post" action="/search">
      <input type="hidden" name="search_type" value="name">
      <label for="name">Full name</label>
      <input id="name" type="text" name="q" placeholder="Jane Doe" required>
      <button type="submit">Search name</button>
    </form>
  </section>
  <details class="opt">
    <summary>Optional: route live lookups through a proxy</summary>
    <p class="muted small">Set <code>OSINT_PROXY</code> before starting the app. This is applied to all searches.</p>
  </details>
  {% if recent %}
  <section class="card">
    <h2>Recent searches</h2>
    <ul class="recent">
      {% for r in recent %}
      <li><a href="/report/{{ r.id }}">{{ r.seed }}</a> <span class="muted">{{ r.when }}</span></li>
      {% endfor %}
    </ul>
  </section>
  {% endif %}
  <footer class="muted">For authorized investigations only.</footer>
</main>
</body>
</html>

```

## webapp/templates/report.html

```

<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>OSINT Investigation Brief - {{ seed }}</title>
<link rel="stylesheet" href="/static/style.css">
</head>
<body>
<div class="topbar no-print">
  <a class="brand" href="/">OSINT</a>
  <div class="actions">
    <a class="btn" href="/history">History</a>
    <button onclick="window.print()">Print / Save as PDF</button>
    <a class="btn" href="/report/{{ id }}/pdf">Download PDF</a>
    <a class="btn" href="/report/{{ id }}/gml">Graph (GML)</a>
  </div>
</div>

<main class="container report">
  <header class="report-header">
    <div class="eyebrow">OSINT / INVESTIGATION BRIEF</div>
    <h1>Public-source findings</h1>
    <p class="seed"><span>Seed identifier</span> <code>{{ seed }}</code></p>
    <p class="muted">Generated {{ when }} <span class="dot">&middot;</span> Report {{ id }}</p>
  </header>

  <section class="summary-grid" aria-label="Report summary">
    <div class="metric"><span class="metric-label">Findings</span><strong>{{ total }}</strong><span class="metric-note">observations c</span>
    <div class="metric"><span class="metric-label">Identifiers</span><strong>{{ identifier_count }}</strong><span class="metric-note"></span>
    <div class="metric"><span class="metric-label">Sources</span><strong>{{ source_count }}</strong><span class="metric-note">distinct
    <div class="metric"><span class="metric-label">Confidence</span><strong>{{ confidence_counts.verified }}</strong><span class="metric-note"></span>
  </section>

  <section class="brief-note">
    <strong>Assessment scope</strong>
    <span>Results reflect public-source observations available at scan time. A finding is not proof of identity; review the linked sou</span>
  </section>

  <section class="card confidence-card">
    <div class="section-heading"><div><span class="section-kicker">01</span><h2>Evidence posture</h2></div><span class="section-caption"></span>
    <div class="confidence-row">
      <div><span class="badge conf-verified">Verified</span><strong>{{ confidence_counts.verified }}</strong><span class="muted">explicit</span>
      <div><span class="badge conf-likely">Likely</span><strong>{{ confidence_counts.likely }}</strong><span class="muted">credible se</span>
      <div><span class="badge conf-unsure">Unsure</span><strong>{{ confidence_counts.unsure }}</strong><span class="muted">collision on</span>
    </div>
  </section>

  {% if source_names %}
  <section class="source-strip"><span class="section-kicker">SOURCE COVERAGE</span>{% for source in source_names %}<span class="source">{{ source }}</span>{% endif %}

  <div class="section-heading standalone"><div><span class="section-kicker">02</span><h2>Identifier map</h2></div><span class="section-caption"></span>
  {% for type, items in groups.items() %}
  <section class="card identifier-card">
    <div class="section-heading"><h3>{{ type|upper }}</h3><span class="section-caption">{{ items|length }} linked</span></div>
    <table>
      <thead><tr><th>Identifier</th><th>Corroboration</th></tr></thead>
      <tbody>
        {% for it in items %}
          <tr><td><code>{{ it.value }}</code></td><td>{{ it.score }}</td></tr>
        {% endfor %}
      </tbody>
    </table>
  </section>
  {% endfor %}

  <section class="card findings-card">
    <div class="section-heading"><div><span class="section-kicker">03</span><h2>Evidence ledger</h2></div><span class="section-caption"></span>
    {% if findings %}
    <table>
      <thead>
        <tr><th>Identifier</th><th>Source</th><th>Confidence</th><th>Link</th><th>Evidence</th></tr>
      </thead>
    </table>
    {% else %}
    <p>No findings were identified during this scan.</p>
    </div>
  </section>

```

```

</thead>
<tbody>
{% for f in findings %}
  <tr>
    <td><code>{{ f.identifier }}</code></td>
    <td>{{ f.source }}</td>
    <td><span class="badge conf-{{ f.confidence }}">{{ f.confidence }}</span></td>
    <td>{% if f.url %}<a href="{{ f.url }}" target="_blank" rel="noopener">open</a>{% else %}&mdash;{% endif %}</td>
    <td class="evidence">
      {% if f.details_summary %}<div class="muted">{{ f.details_summary }}</div>{% endif %}
      {% if f.details %}<details><summary>Inspect source data</summary><pre>{{ f.details | tojson(indent=2) }}</pre></details>{%
    </td>
  </tr>
{% endfor %}
</tbody>
</table>
{% else %}
<p class="muted">No findings for this identifier. Try a different input, or add more sources (HIBP_API_KEY, SERPER_API_KEY, maigre
{% endif %}
</section>

{% if pivots %}
<section class="card pivot-card">
  <div class="section-heading"><div><span class="section-kicker">04</span><h2>Pivot trail</h2></div><span class="section-caption">li
  <ul class="pivots">
    {% for p in pivots %}
      <li><code>{{ p.from }}</code> &#8594; <code>{{ p.to }}</code> <span class="muted">(weight {{ p.weight }})</span></li>
    {% endfor %}
  </ul>
</section>
{% endif %}

<footer class="muted no-print">For authorized investigations only.</footer>
</main>
</body>
</html>

```

**webapp/templates/history.html**

```

<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>OSINT Search History</title>
<link rel="stylesheet" href="/static/style.css">
</head>
<body>
<div class="topbar">
  <a class="brand" href="/">OSINT</a>
  <nav class="actions">
    <a class="btn" href="/">New search</a>
  </nav>
</div>
<main class="container">
  <header class="report-header">
    <h1>Search history</h1>
    <p class="muted">{{ items|length }} saved search{% if items|length != 1 %}es{% endif %}</p>
  </header>

  {% if items %}
  <section class="card">
    <table>
      <thead><tr><th>Seed</th><th>Run at</th><th>Findings</th><th></th></tr></thead>
      <tbody>
        {% for it in items %}
        <tr>
          <td><code>{{ it.seed }}</code></td>
          <td class="muted">{{ it.when }}</td>
          <td>{{ it.total }}</td>
          <td>
            <a href="/report/{{ it.id }}">open</a> &middot;
            <a href="/report/{{ it.id }}/pdf">pdf</a> &middot;
            <a href="/report/{{ it.id }}/gml">gml</a>
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
  </section>
  {% else %}
  <p class="muted">No searches yet. <a href="/">Run one</a>.</p>
  {% endif %}

  <footer class="muted">For authorized investigations only.</footer>
</main>
</body>
</html>

```

## webapp/static/style.css

```

:root {
  --bg: #101316; --card: #171b1d; --card-2: #1d2425; --line: #303a3b;
  --text: #edf1ed; --muted: #9ca9a5; --accent: #8fd3b4;
  --ok: #65d49b; --warn: #e5bd70; --weak: #a4adb0; --ink: #101316;
}
* { box-sizing: border-box; }
body { margin: 0; font-family: Georgia, "Times New Roman", serif; background: radial-gradient(circle at 10% 0%, #1a2625 0, var(--bg) 30%); }
.container { max-width: 920px; margin: 0 auto; padding: 24px; }
.topbar { display: flex; justify-content: space-between; align-items: center; padding: 10px 24px; background: var(--card); border-bottom: 1px solid var(--line); }
.brand { color: var(--text); text-decoration: none; font-weight: 700; letter-spacing: 1px; }
.actions { display: flex; gap: 8px; }
.btn, button { background: var(--accent); color: #fff; border: 0; border-radius: 8px; padding: 8px 14px; font-size: 14px; cursor: pointer; }
.btn:hover, button:hover { filter: brightness(1.1); }
.hero { text-align: center; padding: 60px 0 30px; }
.hero h1 { font-size: 44px; letter-spacing: 5px; margin: 0; }
.tagline { color: var(--muted); }
.search-box { display: flex; gap: 8px; max-width: 660px; margin: 0 auto; }
.search-box input, .search-card input { flex: 1; padding: 12px 14px; border-radius: 10px; border: 1px solid var(--line); background: var(--card); }
.search-box input:focus, .search-card input:focus { outline: 2px solid var(--accent); }
.search-help { text-align: center; margin: 18px 0 10px; }
.search-grid { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 12px; max-width: 660px; margin: 0 auto; }
.search-card { display: flex; flex-direction: column; align-items: stretch; gap: 9px; padding: 15px; background: var(--card); border: 1px solid var(--line); }
.search-card label { font-size: 13px; color: var(--muted); font-weight: 600; }
.search-card button { align-self: flex-start; }
.platform-controls { display: grid; gap: 9px; max-width: 660px; margin: 0 auto 16px; }
.platform-controls label { color: var(--muted); font-size: 13px; font-weight: 600; }
.platform-controls input, .platform-controls select { padding: 12px 14px; border-radius: 10px; border: 1px solid var(--line); background: var(--card); }
.platform-grid { max-width: 660px; margin: 0 auto; display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 12px; }
.platform-button { display: flex; flex-direction: column; align-items: flex-start; gap: 4px; padding: 16px; text-align: left; }
.platform-button small { color: rgba(255,255,255,.72); font-weight: 400; }
.platform-note { max-width: 660px; margin: 18px auto; text-align: center; }
.opt { max-width: 660px; margin: 16px auto 0; color: var(--muted); font-size: 13px; }
.evidence { min-width: 180px; max-width: 420px; }
.evidence details { margin-top: 6px; }
.evidence summary { color: var(--accent); cursor: pointer; }
.evidence pre { white-space: pre-wrap; overflow-wrap: anywhere; max-height: 220px; overflow: auto; margin: 8px 0 0; padding: 9px; border: 1px solid var(--line); }
.card { background: rgba(23,27,29,.92); border: 1px solid var(--line); border-radius: 5px; padding: 20px 22px; margin: 16px 0; box-shadow: 0 4px 12px #000000; }
.card h2, .card h3 { margin: 0; color: var(--text); }
.card h2 { font-size: 18px; letter-spacing: .2px; }
.card h3 { font-size: 14px; letter-spacing: 1.4px; }
table { width: 100%; border-collapse: collapse; font-size: 14px; }
th, td { text-align: left; padding: 8px 10px; border-bottom: 1px solid var(--line); vertical-align: top; }
th { color: var(--muted); font-weight: 600; }
a { color: var(--accent); }
code { background: #0c1112; padding: 3px 7px; border-radius: 3px; font-size: 13px; color: #d7e8df; }
.badge { padding: 3px 9px; border-radius: 999px; font-size: 12px; font-weight: 600; text-transform: capitalize; display: inline-block; }
.conf-verified { background: rgba(46,204,113,.15); color: var(--ok); }
.conf-likely { background: rgba(241,196,15,.15); color: var(--warn); }
.conf-unsure { background: rgba(139,143,168,.15); color: var(--weak); }
.muted { color: var(--muted); font-size: 13px; }
.seed code { font-size: 15px; }
.pivots { margin: 0; padding-left: 18px; line-height: 1.8; }
.recent { list-style: none; padding: 0; }
.recent li { padding: 6px 0; border-bottom: 1px solid var(--line); }
.report-header { padding: 42px 0 24px; border-bottom: 1px solid var(--line); }
.report-header h1 { margin: 8px 0 12px; font-size: 38px; font-weight: 400; letter-spacing: -.5px; }
.eyebrow, .section-kicker, .metric-label { color: var(--accent); font: 600 11px/1.2 "Segoe UI", sans-serif; letter-spacing: 1.6px; text-align: center; }
.seed { margin: 0; color: var(--muted); }
.seed span { margin-right: 8px; font: 600 12px "Segoe UI", sans-serif; text-transform: uppercase; letter-spacing: 1px; }
.dot { color: var(--accent); padding: 0 5px; }
.summary-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 1px; margin: 22px 0; background: var(--line); border: 1px solid var(--line); }
.metric { display: flex; flex-direction: column; gap: 4px; padding: 16px; background: var(--card); }
.metric strong { font-size: 29px; font-weight: 400; }
.metric-note, .section-caption { color: var(--muted); font: 12px "Segoe UI", sans-serif; }
.brief-note { display: flex; gap: 16px; padding: 14px 16px; margin: 18px 0 26px; border-left: 3px solid var(--accent); background: rgba(23,27,29,.92); }
.brief-note strong { color: var(--text); white-space: nowrap; }
.section-heading { display: flex; justify-content: space-between; align-items: center; gap: 12px; margin-bottom: 14px; }
.section-heading > div { display: flex; align-items: center; gap: 11px; }
.section-heading .section-kicker { color: var(--muted); }
.section-heading.standalone { margin: 28px 0 12px; }
.section-heading.standalone h2 { font-size: 20px; font-weight: 400; }
.source-strip { display: flex; flex-wrap: wrap; align-items: center; gap: 7px; margin: 18px 0 28px; font: 12px "Segoe UI", sans-serif; }
.source-chip { padding: 5px 9px; border: 1px solid var(--line); border-radius: 3px; color: var(--muted); }

```

```

.confidence-row { display: grid; grid-template-columns: repeat(3, 1fr); gap: 12px; }
.confidence-row > div { display: grid; grid-template-columns: auto 1fr; gap: 5px 9px; align-items: center; }
.confidence-row strong { font-size: 20px; font-weight: 400; }
.confidence-row .muted { grid-column: 1 / -1; }
.identifier-card { margin: 8px 0; }
footer { text-align: center; padding: 20px 0; }
@media print {
  .no-print { display: none !important; }
  body { background: #fff; color: #000; }
  body { font-family: Georgia, "Times New Roman", serif; }
  .card, .metric { background: #fff; border: 1px solid #ccc; break-inside: avoid; box-shadow: none; }
  .summary-grid { background: transparent; border: 0; gap: 8px; }
  .brief-note { background: #f4f7f5; color: #333; }
  code { background: #f0f0f0; color: #000; }
  a { color: #000; text-decoration: none; }
  .badge { border: 1px solid #999; }
  .conf-verified, .conf-likely, .conf-unsure { background: #fff; }
  table, th, td { border-color: #ccc; }
}
@media (max-width: 620px) {
  .summary-grid, .confidence-row { grid-template-columns: repeat(2, 1fr); }
  .brief-note { flex-direction: column; gap: 4px; }
  .report-header h1 { font-size: 31px; }
  .search-grid { grid-template-columns: 1fr; }
  .platform-grid { grid-template-columns: 1fr; }
  .search-box { flex-direction: column; }
  .search-box button { align-self: stretch; }
  .topbar { padding: 10px 14px; }
  .actions { flex-wrap: wrap; justify-content: flex-end; }
}

```

## requirements.txt

```
# Core ? required for the base pipeline (orchestrator, native sources, reporting).
httpx[http2]==0.28.1
phonenumbers==9.0.39
networkx==2.8.8
python-dotenv==1.2.3

# Web application and PDF export
fastapi
uvicorn[standard]
jinja2
python-multipart
fpdf2

# Optional ? only needed if you use the corresponding source.
# Comment out what you don't need. socialscan/telethon aren't pinned here ?
# they weren't in the captured environment, so check PyPI for current versions
# before installing.
maigret==0.6.5 # sources/wrappers.py : MaigretSource (username sweep)
socialscan    # sources/wrappers.py : SocialscanSource (email/username availability)
telethon      # sources/telegram_phone.py : TelegramPhoneSource

# Dev / testing
pytest==8.3.3
pytest-asyncio==0.24.0
```

## build\_pdf.py

```

"""python build_pdf.py -> OSINT_Blueprint.pdf (blueprint + all source files)"""
from pathlib import Path

from fpdf import FPDF

CORE = ["osint/models.py", "osint/normalizers.py", "osint/graph.py",
        "osint/confidence.py", "osint/reporting.py", "osint/orchestrator.py",
        "osint/__main__.py"]
SRC = ["osint/sources/base.py", "osint/sources/native.py",
        "osint/sources/wrappers.py", "osint/sources/dorks.py",
        "osint/sources/telegram_phone.py", "osint/sources/breaches.py"]
WEB = ["run_web.py", "webapp/server.py", "webapp/store.py", "webapp/pdf_export.py",
        "webapp/templates/index.html", "webapp/templates/report.html",
        "webapp/templates/history.html", "webapp/static/style.css"]

BLUEPRINT = """OSINT PRODUCT BLUEPRINT
=====

PIPELINE: identifier -> normalize -> parallel source queries -> correlation graph -> confidence -> report
PIVOTING: every finding seeds new queries (email -> username -> name -> phone), max_depth=2.

TECHNIQUE MATRIX
- Email: registration enumeration (user-scanner / socialscan), breaches (HIBP, DeHashed),
  Gravatar JSON, Google-account surface (Epieos/GHunt)
- Phone: E.164 + carrier/line-type (phonenumbers), enrichment (Twilio Lookup, Numverify),
  WhatsApp/Telegram presence, breach records
- Name: search-engine x-ray (Serper, site: dorks), people-data APIs (PDL/Apollo)
- Username: Mairgret (3000+ sites), Sherlock, Blackbird

BREACHES: HIBP v3 via hibp-api-key (free at haveibeenpwned.com). 200 = breaches found,
404 = clean. Add more engines (DeHashed/Snubase/IntelX) behind the same Source API.

WEB APP (FastAPI + Jinja2)
python run_web.py          -> http://127.0.0.1:8000 (opens browser automatically)
POST /search               -> runs the pipeline, saves report, redirects to result page
                           (optional proxy field, or OSINT_PROXY env var)
GET /history               -> every saved search (open / pdf / gml links)
GET /report/{id}           -> results page (identifiers, findings, confidence, pivots)
GET /report/{id}/pdf       -> server-generated PDF download
GET /report/{id}/gml       -> correlation graph as GML (opens in Gephi / yEd)
Browser Print / Save-as-PDF -> printable layout via print CSS
Reports persist as JSON+GML under reports/ (recent on home page, full history page).

PROXY: per-request field on the search form, or OSINT_PROXY=http://user:pass@host:port
environment variable. Applies to every httpx-backed live source via the shared client.
Subprocess-only sources (maigret, socialscan) bypass it; run those over a system/WSL proxy.

CONFIDENCE MODEL
VERIFIED - target server explicitly confirmed (HTTP 200 profile, reset-flow response)
LIKELY   - corroborated search hit / single source
UNSURE   - collision-prone (esp. names)

ENGINEERING: async httpx + per-source rate limits, proxy rotation, module health-checks
(module rot is the #1 failure mode), graph persistence (JSON + GML), Redis caching.

COMPLIANCE: GDPR/CCPA controller duties, FCRA ToS prohibition on employment/credit/
tenant screening, platform-ToS review of enumeration modules. For authorized use only.

USAGE
pip install -r requirements.txt
pipx install maigret          # optional username sweep (3000+ sites)
set HIBP_API_KEY=...         # optional breach module
set SERPER_API_KEY=...       # optional search x-ray
set OSINT_PROXY=http://user:pass@host:port # optional proxy for live lookups
python run_web.py            # start the web app
python -m osint "john.doe@gmail.com" # or CLI mode
python build_pdf.py          # regenerate this document
"""

HEADER = "OSINT Blueprint - privileged/internal"

class Doc(FPDF):

```

```

def header(self):
    self.set_font("Courier", size=7)
    self.cell(0, 4, HEADER, align="C", new_x="LMARGIN", new_y="NEXT")
    self.ln(2)

def code_block(pdf: FPDF, text: str):
    pdf.set_font("Courier", size=7)
    pdf.set_fill_color(245, 245, 245)
    for line in text.splitlines():
        if pdf.get_y() > 270:
            pdf.add_page()
        safe = line[:150].encode("latin-1", "replace").decode("latin-1")
        pdf.cell(0, 3.4, safe, fill=True, new_x="LMARGIN", new_y="NEXT")

def main() -> None:
    pdf = Doc(unit="mm", format="A4")
    pdf.set_auto_page_break(True, margin=15)
    pdf.add_page()
    pdf.set_font("Helvetica", "B", 16)
    pdf.cell(0, 10, "OSINT Blueprint", new_x="LMARGIN", new_y="NEXT")
    pdf.ln(2)
    code_block(pdf, BLUEPRINT)

    for path in CORE + SRC + WEB + ["requirements.txt", "build_pdf.py"]:
        p = Path(path)
        if not p.exists():
            continue
        pdf.add_page()
        pdf.set_font("Helvetica", "B", 11)
        pdf.cell(0, 6, path.encode("latin-1", "replace").decode("latin-1"), new_x="LMARGIN", new_y="NEXT")
        pdf.ln(1)
        code_block(pdf, p.read_text(encoding="utf-8"))

    pdf.output("OSINT_Blueprint.pdf")
    print("[+] wrote OSINT_Blueprint.pdf")

if __name__ == "__main__":
    main()

```